OpenTransformer: Building and Verifying a Tiled 4×4 Systolic Array Accelerator

Ibraheem Syed
Independent Hardware Design Project
September 2026

***Abstract*—This paper presents OpenTransformer, an independent hardware design project focused on building a small matrix-multiply accelerator in SystemVerilog. The current milestone uses a 4×4 physical systolic array with 16 processing elements. Instead of building a full physical 16×16 systolic array with 256 processing elements, the design reuses the smaller 4×4 array through tiling to complete a 16×16 matrix multiplication workload.**

The design was built from smaller RTL blocks, including a multiply-accumulate unit, a processing element, a 4×4 processing-element array, a tile controller, and a top-level 16×16 matrix multiplication module. The final top-level design passed Verilator simulation with all 256 output values correct, passed Verilator lint with no warnings or errors in the final lint log, synthesized successfully with Yosys, and produced FPGA artifacts for the Tang Nano 9K platform.

The project also includes OpenROAD physical design evidence. The full 16×16 top-level design was attempted in OpenROAD but did not complete the full flow. The 4×4 PE-array compute core did complete the OpenROAD flow on Nangate45 through routing, final reports, and GDS generation. The final routed PE-array core had a reported design area of 14,764 μm^2 at 31% utilization, 0 detailed-route violations, 0 antenna net violations, 0 antenna pin violations, 113,082 μm of routed wire length, and 52,211 vias. This paper documents the project by separating the completed FPGA top-level result from the completed OpenROAD PE-array core result.

***Index Terms*—SystemVerilog, FPGA, OpenROAD, RTL design, systolic array, matrix multiplication, Verilator, Yosys, hardware acceleration.**

I. Introduction

Matrix multiplication is one of the main operations used in modern computing. It appears in graphics, signal processing, scientific computing, and machine learning. In neural networks and transformer models, a lot of the work eventually becomes repeated multiply-and-accumulate operations. Because of that, matrix multiplication is a good starting point for learning how AI-style hardware accelerators work.

OpenTransformer is an independent hardware project that I built to understand how matrix multiplication can be mapped into actual digital hardware. I did not start with a complete accelerator. I started with smaller building blocks such as basic logic, an ALU, a register file, a finite-state-machine controller, a multiply-accumulate unit, and processing elements. After those smaller blocks worked, I connected them into a 4×4 systolic array and then built a tiled 16×16 matrix multiplication design around it.

The main idea in this milestone is hardware reuse. A full physical 16×16 systolic array would need 256 processing elements. That would be much larger. OpenTransformer uses a 4×4 physical systolic array with 16 processing elements. The larger 16×16 workload is completed by reusing that smaller array across multiple tiles.

This difference matters. The design does not physically contain a 16×16 array. It physically contains a 4×4 array, and the tile controller reuses that array over time. This paper explains the architecture, how the tiling works, how the RTL was tested, and what evidence was produced from simulation, FPGA implementation, and OpenROAD.

The final milestone includes two main implementation results. The first is the full top-level FPGA-oriented design for 16×16 matrix multiplication. The second is the OpenROAD physical-design result for the 4×4 PE-array compute core. The OpenROAD result is not for the full top-level design. It is for the compute core, which is still important because that is the actual 16-PE systolic array used in the design.

II. Background

Matrix multiplication can be written as

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} B_{k,j} \tag{1}$$

Each value in output matrix C is created by multiplying one row from matrix A with one column from matrix B and adding all the products. This looks simple as an equation, but the hardware design has to handle data movement, timing, storage, and control.

The basic operation is multiply-and-accumulate. A multiply-accumulate unit, or MAC, multiplies two numbers and adds the product into an accumulator. This is useful for matrix multiplication because every output element is a sum of products.

A systolic array connects many processing elements together in a grid. Each processing element does a small piece of the computation and passes data to its neighbors. In this project, A values move horizontally across rows and B values move vertically through columns. Each PE uses the data passing through it to update a local partial sum.

A larger systolic array can compute more work in parallel, but it also uses more hardware. A smaller array uses fewer resources, but it has to be reused across more cycles. This is where tiling becomes useful. Instead of building a physical array for the whole matrix, the matrix is split into smaller tiles, and the same hardware is reused for each tile.

III. Project Goal

The goal of OpenTransformer was not just to write code that looks like hardware. The goal was to build a design and collect real evidence from tools. For this milestone, the target was:

- Build a 4×4 physical systolic array in SystemVerilog.
- Use the array as the compute core for a tiled 16×16 matrix multiplication design.
- Verify the full 16×16 result in simulation.
- Run lint and synthesis.
- Generate FPGA artifacts for Tang Nano 9K.
- Run OpenROAD physical design on the 4×4 PE-array core.
- Save logs, reports, layouts, hashes, and final evidence files.

This project was also meant to teach the full hardware flow. Simulation is important, but it is not the whole process. A design also needs linting, synthesis, implementation reports, and saved artifacts. The project became more realistic once the outputs from the tools were treated as the real source of truth.

IV. Architecture Overview

The OpenTransformer design is built as a hierarchy of SystemVerilog modules. Fig. 1 shows the high-level architecture. The main design files are summarized in Table I.

TABLE I
Main RTL Modules

Module	Purpose
`mac.sv`	Multiply-accumulate unit
`pe.sv`	Processing element
`pe_array_4x4.sv`	Physical 4×4 systolic array
`tile_controller.sv`	Controls tiled execution
`matmul_16x16.sv`	Top-level tiled matrix multiply
`fpga_top.sv`	FPGA wrapper

The lowest-level arithmetic block is the MAC. The PE wraps the MAC and adds the data movement needed for the systolic array. The 4×4 array connects 16 PEs together. The tile controller handles which part of the 16×16 matrix is being computed. The top-level module connects the controller, matrix storage, array, and result readout.

This hierarchy made the project easier to build because each block had a clear job. It also made debugging easier. A problem could be narrowed down to the MAC, PE, array, controller, or top-level result path.

V. Processing Element Datapath

The processing element is the basic compute unit of the systolic array. Fig. 2 shows the PE datapath.

Each PE receives an A input and a B input. The A value is passed to the right, and the B value is passed downward. At the same time, the PE multiplies the local A and B values and adds the product into an accumulator.

The accumulator is wider than the input values because each output value is the sum of many products. In this project, the design uses INT8-style inputs and a wider accumulation path. This matches the basic idea used in many quantized accelerator designs, where smaller input values are multiplied and accumulated into a larger result.

The PE is small by itself, but the full array is built by repeating it. A 4×4 array has 16 PEs. This also matches the FPGA result, where the final resource extraction showed 16 multiplier blocks for the physical compute array.

VI. Tiled Matrix Multiplication

The main design choice in this project is tiling. A 16×16 output matrix contains 256 output values. Building a physical 16×16 systolic array would mean building 256 processing elements. OpenTransformer instead uses a 4×4 physical array with 16 processing elements.

The 16×16 output matrix is divided into 4×4 output tiles. The same physical 4×4 array computes one tile at a time. After one tile is finished, the controller moves to the next tile. This continues until all output tiles are finished.

Since the 16×16 matrix has four 4-row groups and four 4-column groups, there are 16 output tiles total. For each output tile, the design also has to move through the inner dimension of the multiplication. This is why the controller is important. It has to keep track of row tiles, column tiles, and the partial sums.

The basic process is:

1) Select the current 4×4 output tile.
2) Feed the needed A values across the rows.
3) Feed the needed B values down the columns.
4) Let the PE array multiply and accumulate.
5) Preserve partial sums when needed.
6) Store the completed output tile.
7) Repeat until the full 16×16 output matrix is complete.

This design is slower than a full physical 16×16 array because it reuses hardware over time. However, it uses far fewer processing elements. For this project, that tradeoff made sense because the goal was to build a complete working design and fit the implementation into a small learning-focused hardware flow.

VII. RTL Implementation

The design was written in SystemVerilog. The project was built in stages. Earlier stages focused on smaller blocks, and later stages connected those blocks into a larger accelerator.

The MAC was built first as the arithmetic block. Then the PE was built around the MAC. After that, 16 PEs were connected into a 4×4 systolic array. The tile controller and top-level matrix multiply module were added after the compute array was working.

The main RTL source files are:

- `rtl/mac.sv`
- `rtl/pe.sv`
- `rtl/pe_array_4x4.sv`
- `rtl/tile_controller.sv`

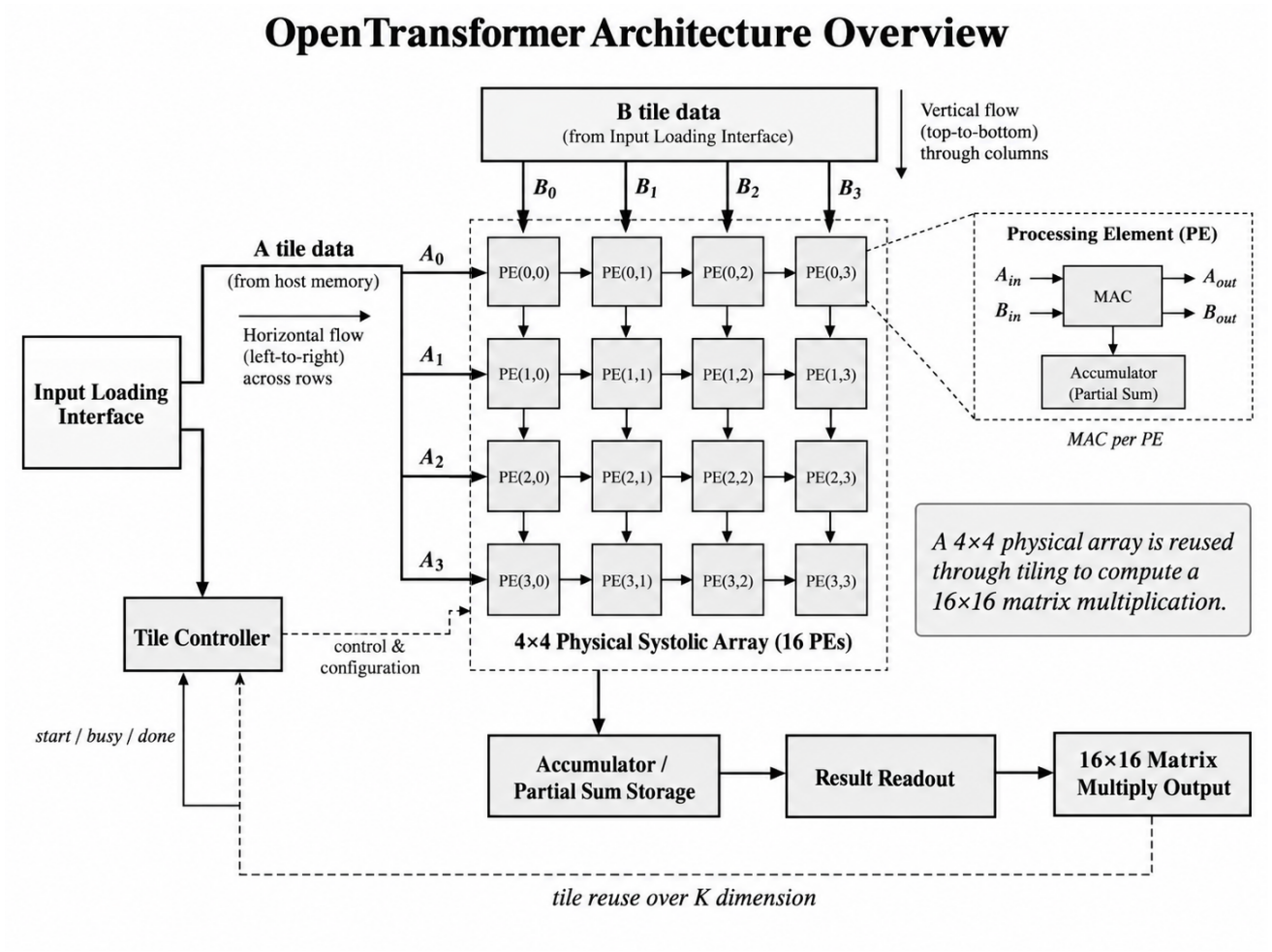

Fig. 1. OpenTransformer architecture overview. The design uses a 4×4 physical systolic array with 16 processing elements. A tile controller reuses this smaller array to complete a 16×16 matrix multiplication workload.

- `rtl/matmul_16x16.sv`
- `rtl/fpga_top.sv`

The final full-design testbench is:

- `tb/tb_matmul_16x16.sv`

This testbench loads matrix data, starts the design, waits for completion, and then checks the result values. The most important part is that it checks every output value, not just a done signal.

The Yosys synthesis hierarchy also confirmed that the intended structure was present. The top-level module was `matmul_16x16`. Under it, the design contained the 4×4 PE array and the tile controller. Inside the PE array were 16 processing elements, and each processing element contained a MAC datapath.

VIII. Verification

Verification was done with Verilator. The final simulation tested the complete 16×16 matrix multiplication path. The testbench checked all 256 output values.

The final simulation log reported:

TABLE II
Final Top-Level Simulation Result

Item	Result
Design tested	`matmul_16x16`
Matrix size	16×16
Output values checked	256
Correct output values	256
Incorrect output values	0
Final status	PASS

```
16x16 MATRIX MULTIPLY TEST PASS
All 256 outputs correct
```

This matters because it proves the top-level tiled design worked for the directed test case. It also proves that the controller, array, partial-sum behavior, and output readout worked together for a complete matrix result.

The verification result should still be described honestly. This was a directed test, not a full formal proof and not a complete randomized verification environment. A future version should test random matrices, negative values, zero

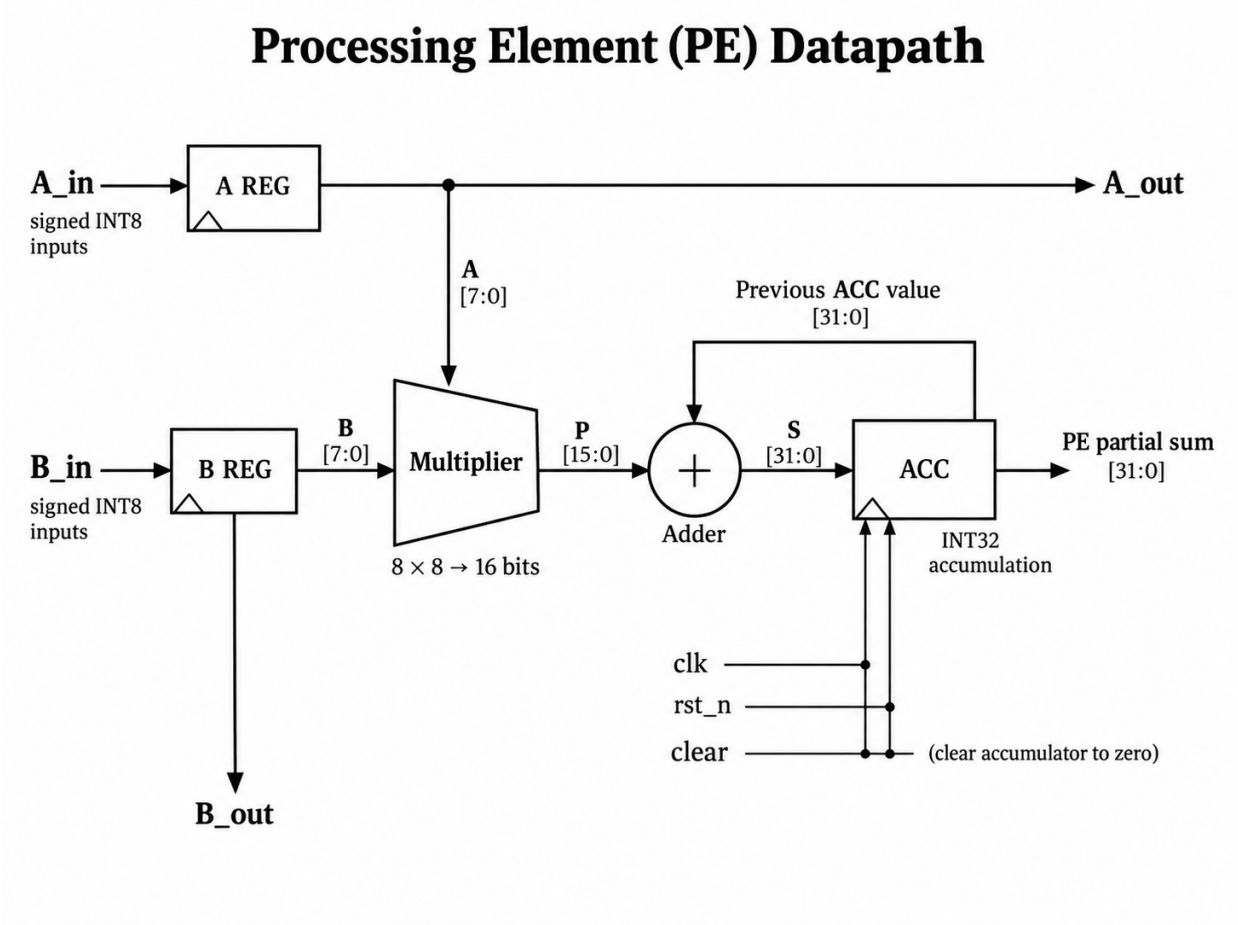

Fig. 2. Processing element datapath. Each PE receives INT8-style input values, forwards A and B values through the array, multiplies them, and accumulates the result into a wider accumulator.

matrices, repeated operations, reset during operation, and edge cases.

IX. Lint and RTL Synthesis

Final lint was performed with Verilator. The final lint log did not report warnings or errors. This helped check that the RTL was clean before synthesis.

Yosys synthesis was also run on the top-level matrix multiplication design. The final Yosys statistics are shown in Table III.

TABLE III
Final Yosys Design Statistics for `matmul_16x16`

Metric	Value
Wires	5,983
Wire bits	58,258
Public wires	1,169
Public wire bits	16,341
Memories	0
Memory bits	0
Processes	0
Cells	38,170

Yosys reported warnings related to replacing internal array-style bus structures with registers. These warnings came from internal signals in the 4×4 array. They did not stop synthesis and did not represent a functional test failure. They showed how the tool converted SystemVerilog structures into lower-level logic.

X. FPGA Implementation

The full `matmul_16x16` design produced FPGA artifacts for the Tang Nano 9K platform. The saved FPGA evidence includes the bitstream, synthesis JSON, place-and-route JSON, constraint file, hashes, and summary.

The main FPGA evidence files are:

```
reports/final/opent_final.fs
reports/final/opent_synth_final.json
reports/final/opent_pnr_final.json
reports/final/tangnano9k_final.cst
reports/final/final_fpga_summary.md
reports/final/final_fpga_hashes.txt
```

Resource counts extracted from the FPGA place-and-route JSON are shown in Table IV.

TABLE IV
FPGA RESOURCE COUNTS

Resource Group	Count
LUT group	4,703
DFF group	843
RAM16SDP4	168
MULT9X9	16
IO	4

The 16 `MULT9X9` blocks are important because they match the 16 PEs in the physical 4×4 array. This is another piece of evidence that the compute fabric is a 4×4 physical array, not a physical 16×16 array.

XI. OPENROAD PHYSICAL DESIGN

The OpenROAD part of the project was done after the FPGA and simulation evidence. This was important because the paper should not claim physical-design results before the physical-design flow was actually run.

There were two OpenROAD attempts. The first was for the full `matmul_16x16` top-level design. That run reached clock tree synthesis, but it did not complete the full OpenROAD flow. The run ended because the OpenROAD Docker process hit an illegal-instruction crash. Because of that, the full top-level OpenROAD result is only partial and should not be described as a completed routed ASIC result.

The completed OpenROAD result is for the 4×4 PE-array compute core, `pe_array_4x4`. This is the real physical compute array used in the accelerator. The PE-array core completed the OpenROAD flow through synthesis, floorplanning, placement, CTS, global routing, detailed routing, filler insertion, RC extraction, final report generation, and GDS generation.

Fig. 3 shows the routed OpenROAD layout view.

TABLE V
OPENROAD PHYSICAL RESULTS FOR `PE_ARRAY_4X4`

Metric	Result
Platform	Nangate45
Clock target	10 ns
Final design area	14,764 μm^2
Final utilization	31%
Detail-route violations	0
Antenna net violations	0
Antenna pin violations	0
Total routed wire length	113,082 μm
Total vias	52,211
Final GDS generated	Yes

The OpenROAD result is useful because it shows that the 4×4 compute core is not only a simulated RTL block. It was also taken through a physical-design flow and produced routed layout artifacts.

XII. CLOCK TREE AND TIMING

Clock tree synthesis was an important part of the OpenROAD run. In the manual CTS check, the clock tree for the PE-array core started with 704 clock sinks. OpenROAD clustered the sinks and created 146 clock buffers and 146 clock nets. The maximum clock-tree level was 4.

TABLE VI
MANUAL CTS RESULTS FOR `PE_ARRAY_4X4`

Metric	Result
Original clock sinks	704
Clustered sinks	129
Created clock buffers	146
Created clock nets	146
Max clock-tree level	4
Worst max slack	8.82 ns
Worst min slack	0.12 ns
TNS max	0.00
TNS min	0.00

The timing numbers show that the PE-array core met the 10 ns clock target at the CTS stage. The global-route report also showed a reported clock period of 1.149 ns and slack of 8.790 ns. These values are evidence from the tool flow, but they should be interpreted as results for the PE-array core only.

XIII. ROUTING, IR, AND CELL REPORT

The detailed router completed with 0 violations after optimization. During routing, the violation count decreased across iterations until the final detailed routing pass reached 0 violations. The final routed wire length was 113,082 μm and the total number of vias was 52,211.

The final report also included IR-drop information. The VDD report showed a worst-case IR drop of 1.83e-03 V, or 0.17%. The VSS report showed a worst-case IR drop of 2.92e-03 V, or 0.27%. The total reported power was 6.48e-03 W.

TABLE VII
OPENROAD IR AND POWER RESULTS

Metric	VDD	VSS
Total power	6.48e-03 W	6.48e-03 W
Worst-case IR drop	1.83e-03 V	2.92e-03 V
Percentage drop	0.17%	0.27%

The cell type report is shown in Table VIII. The largest area contribution came from fill cells, which are part of completing the physical layout. The active logic included sequential cells, combinational cells, clock buffers, timing repair buffers, and inverters.

XIV. EVIDENCE AND REPRODUCIBILITY

A major goal of this project was to save evidence instead of only describing results. For hardware projects, it is easy to say that something works. It is better to save the exact logs, reports, and generated files that prove what happened.

The final evidence is organized in the repository. Table IX lists the most important files.

The OpenROAD evidence was committed after the PE-array physical flow completed. The commit added the OpenROAD

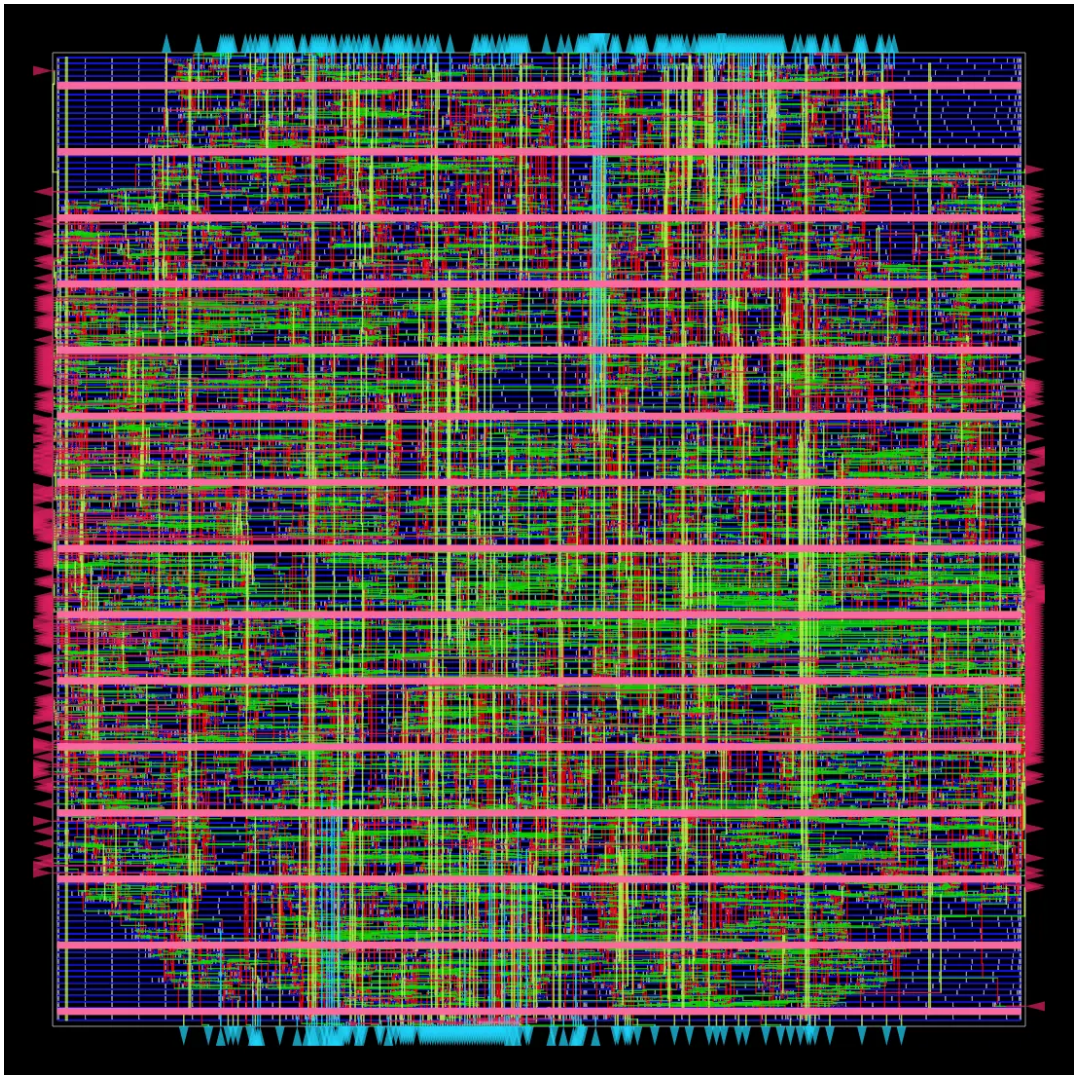

Fig. 3. Final OpenROAD routed layout view of the 4×4 PE-array compute core on the Nangate45 platform. This is physical-design evidence for the compute core, not the full 16×16 top-level controller design.

TABLE VIII
OPENROAD CELL TYPE REPORT

Cell Type	Count	Area
Fill cell	12,843	32,998.36
Tap cell	391	104.01
Clock buffer	200	239.93
Timing repair buffer	617	517.37
Inverter	290	154.81
Clock inverter	61	41.23
Sequential cell	704	3,745.28
Multi-input combinational cell	5,639	9,960.90
Total	20,745	47,761.90

TABLE IX
FINAL EVIDENCE FILES

File or Folder	Purpose
`reports/final/`	FPGA and top-level logs
`final_sim_16x16.log`	16×16 simulation result
`final_lint.log`	Verilator lint result
`final_yosys_synth.log`	Yosys synthesis result
`opent_final.fs`	FPGA bitstream
`opent_pnr_final.json`	FPGA PNR JSON
`reports/openroad/`	OpenROAD evidence
`OPENROAD_SUMMARY.md`	OpenROAD summary
`6_final.gds`	Final PE-array GDS
`6_final.def`	Final PE-array DEF
`SHA256SUMS.txt`	Artifact hashes

configs, reports, final routed files, final GDS, final DEF, summary file, and hashes.

This evidence-based workflow helped keep the project honest. The paper reports the full top-level FPGA result because the full top-level design was simulated and implemented for FPGA. The paper reports the completed OpenROAD result only for the PE-array core because that is the part that completed routing and GDS generation in OpenROAD.

XV. Discussion

The biggest thing I learned from this project is that hardware design is more than writing RTL. A design has to pass through many steps before the result is meaningful. Simulation checks behavior. Lint catches common RTL issues. Synthesis checks whether the code can become hardware. FPGA implementation and OpenROAD physical design give more evidence about how the hardware maps into real structures.

Another lesson was that control logic is just as important as datapath logic. The MAC operation is simple by itself, but the full accelerator also needs tile control, result storage, output selection, and timing. Many of the hard parts came from making sure values were checked at the right time and that the controller moved through the matrix correctly.

The project also showed why wording matters. Saying "16×16 systolic array" would be misleading if someone thinks that means 256 physical processing elements. The accurate description is that the workload is 16×16, but the physical array is 4×4. The larger matrix is handled through tiling.

The OpenROAD work also changed the project. Before running OpenROAD, it would have been wrong to claim physical-design results. After the flow completed for the PE-array core, the project had actual routed layout and GDS evidence. At the same time, the full top-level OpenROAD run did not complete, so that result has to be described as partial. This is still useful because failed or partial tool runs are part of real engineering.

XVI. Limitations

This milestone has several limitations.

First, the final full top-level simulation is a directed test. It checks all 256 output values for one full 16×16 case, but it does not prove every possible input. Future verification should include random matrices, negative values, zero values, identity-like matrices, overflow cases, reset behavior, and repeated start/done transactions.

Second, the FPGA implementation was focused on producing working artifacts and resource evidence. It does not yet include a detailed performance study with latency, throughput, or maximum measured frequency.

Third, the completed OpenROAD result is for the 4×4 PE-array compute core, not the full `matmul_16x16` top-level design. The full top-level design was attempted in OpenROAD but did not complete the full route/signoff flow. This paper does not claim that the full top-level 16×16 design has a completed ASIC layout.

Fourth, Nangate45 is used as an OpenROAD educational/platform flow. It is useful for learning physical design, but the numbers should not be treated as final product silicon results.

XVII. Future Work

Future work should start with stronger verification. The testbench should include randomized matrices and edge cases. It should also run multiple operations back to back to make sure the accelerator returns to a correct state after finishing one matrix multiplication.

The design should also measure performance. Important numbers would include cycle count, latency per tile, total latency for 16×16 matrix multiplication, and throughput. These numbers would make it easier to compare the 4×4 tiled design with larger physical arrays.

The memory and input/output interface can also be improved. The current design focuses on proving the compute and tiling concept. A stronger accelerator would need better buffering, clearer host communication, and a more realistic data-loading path.

The full `matmul_16x16` top-level OpenROAD flow should also be revisited. The design may need changes to memory structure, top-level interface, or physical-design configuration before it can complete cleanly through routing and signoff. A future paper could compare the PE-array core layout with a full top-level layout.

Longer term, OpenTransformer can move closer to transformer workloads. Matrix multiplication is the foundation, but future work could add quantization experiments, activation functions, memory scheduling, and more complete accelerator datapaths.

XVIII. Conclusion

This paper presented OpenTransformer, a tiled matrix-multiply accelerator project built in SystemVerilog. The current milestone uses a 4×4 physical systolic array with 16 processing elements and reuses that array to compute a 16×16 matrix multiplication workload.

The full top-level design passed Verilator simulation with all 256 output values correct, passed final lint, synthesized with Yosys, and produced FPGA artifacts for the Tang Nano 9K platform. The FPGA resource evidence showed 16 multiplier blocks, matching the 16 PEs in the physical 4×4 compute array.

The project also produced OpenROAD physical-design evidence for the 4×4 PE-array core. That core completed the OpenROAD flow through detailed routing, final reports, and GDS generation on Nangate45. The final routed core had 14,764 μm^2 design area at 31% utilization, 0 detail-route violations, 0 antenna violations, 113,082 μm routed wire length, and 52,211 vias.

The final result is a complete and documented milestone: a verified tiled matrix-multiply accelerator with FPGA evidence and a completed routed OpenROAD layout for the compute core. The most important part is that the claims are tied to saved evidence, and the paper clearly separates what completed from what remains future work.

References

[1] The OpenROAD Project, "OpenROAD-flow-scripts."
[2] YosysHQ, "Yosys Open SYnthesis Suite."
[3] Verilator, "Verilator SystemVerilog simulator and lint tool."
[4] Nangate, "Nangate Open Cell Library."
[5] I. Syed, "OpenTransformer final RTL, FPGA, and OpenROAD evidence," project repository, September 2026.